🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 5 Comprehensive Post-Assessment (50 MCQs)

Course: B.Tech CSE 7th Sem — High Performance Computing Masterclass
Module: Day 5 Final Mastery Exam: Generative AI, LLMs, Intel AMX, Habana Gaudi 2/3, OpenVINO IR, NNCF INT4 & Distributed LoRA
Total Questions: 50 | **Duration**: 60 Minutes | **Maximum Marks**: 50

📋 Master Evaluation Key & Forensic Solution Guide (Instructor Copy)

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
01	B	`Analyze`	A dual-socket 4th Gen Intel Xeon Scalable 8480+ se...
02	B	`Understand`	What is the physical architectural structure of th...
03	B	`Knowledge`	In the Habana Gaudi 2 architecture, what is the ex...
04	B	`Analyze`	How does the Habana Gaudi 3 architecture improve c...
05	B	`Understand`	In OpenVINO Intermediate Representation (IR), what...
06	B	`Apply`	When quantizing a pre-trained transformer model us...
07	A	`Apply`	Consider a linear layer with input dimension $d_{i...
08	B	`Apply`	In a production Slurm cluster with 4 compute nodes...
09	B	`Knowledge`	What is the primary operational purpose of the Int...
10	B	`Analyze`	Why is the prompt processing phase ('prefill') in ...
11	B	`Understand`	In the OpenVINO GenAI C++/Python API (`openvino_ge...
12	B	`Evaluate`	When deploying an LLM with INT4 Weight-Only quanti...
13	B	`Knowledge`	What is the primary role of the `TDPBF16PS` instru...
14	B	`Understand`	In distributed LLM fine-tuning using DeepSpeed ZeR...
15	B	`Analyze`	What is the key advantage of Habana's integrated o...
16	B	`Understand`	In continuous batching LLM serving architectures, ...
17	A	`Evaluate`	In the context of Intel NNCF, what is the effect o...
18	B	`Apply`	In a FastAPI microservice serving an OpenVINO LLM,...
19	B	`Understand`	What is the role of the `alpha` hyperparameter in ...
20	A	`Apply`	Why does a 13-billion parameter FP16 model require...

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
21	B	`Understand`	In Habana SynapseAI, what does the environment var...
22	B	`Analyze`	What is the primary difference between Hugging Fac...
23	B	`Knowledge`	How does Intel AMX accelerate INT8 quantization wo...
24	B	`Knowledge`	In Slurm job submission, what is the role of the e...
25	B	`Evaluate`	What is the technical cause of 'Perplexity Degrada...
26	B	`Understand`	In OpenVINO runtime, what is the primary benefit o...
27	B	`Knowledge`	In Habana Gaudi's SynapseAI stack, what is the 'TP...
28	A	`Analyze`	What is the primary difference between GPTQ (Gener...
29	B	`Understand`	In an LLM benchmark report, what does the metric '...
30	B	`Understand`	What is 'FlashAttention' and what core hardware ar...
31	B	`Analyze`	In a dual-socket Intel Xeon system with 64 cores p...
32	B	`Understand`	In distributed LLM training, what is the role of '...
33	B	`Knowledge`	In the Slurm workload manager, what does the comma...
34	B	`Analyze`	What is the primary motivation for using Asymmetri...
35	B	`Knowledge`	What is the purpose of the `hl-smi -q` command in ...
36	B	`Analyze`	In a distributed deep learning cluster, what is th...
37	B	`Knowledge`	What is the primary role of the `AutoTokenizer` cl...
38	B	`Understand`	In the context of the Roofline Model, what does th...
39	B	`Analyze`	How does Intel NNCF implement 'SmoothQuant' to ena...
40	B	`Understand`	In Habana Gaudi systems, what does the term 'HBM2e...
41	A	`Knowledge`	What is the primary function of the OpenVINO Model...
42	B	`Understand`	In a distributed training job using PyTorch FSDP (...
43	A	`Analyze`	What is the primary microarchitectural advantage o...
44	B	`Evaluate`	In OpenVINO INT4 model serving, why is the memory ...
45	A	`Knowledge`	Which of the following Slurm environment variables...
46	B	`Understand`	In LLM generation sampling, what is the role of th...
47	B	`Analyze`	What is the primary reason why 'Continuous Batchin...
48	B	`Apply`	When converting a PyTorch model to OpenVINO IR usi...
49	B	`Analyze`	What is the primary technical distinction between ...
50	B	`Evaluate`	In an enterprise HPC cluster deployment, what is t...

🔬 Complete Questions with In-Depth Architectural Explanations

Question 01

A dual-socket 4th Gen Intel Xeon Scalable 8480+ server has 112 physical cores and 8 DDR5-4800 memory channels delivering 307.2 GB/s theoretical peak bandwidth. When running autoregressive token generation on a 7-billion parameter LLM with INT4 weight quantization (model weight size = 3.5 GB), assuming an 80% memory bandwidth efficiency factor and batch size = 1, what is the maximum theoretical generation throughput in Tokens Per Second (TPS)?

- **(a)** 12.5 TPS
- **(b) [CORRECT]** 70.2 TPS (calculated as (307.2 * 0.80) / 3.5 GB)
- **(c)** 350.0 TPS
- **(d)** 1,024.0 TPS

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: In memory-bandwidth-bound decode at batch size 1, generating each token requires streaming the entire weight set across the memory bus once. Effective bandwidth = 307.2 GB/s * 0.80 = 245.76 GB/s. Max theoretical tokens/sec = 245.76 GB/s / 3.5 GB/token ≈ 70.22 tokens/second.

Question 02

What is the physical architectural structure of the Intel Advanced Matrix Extensions (AMX) TMUL engine in 4th/5th Gen Intel Xeon processors?

- **(a)** A 1D vector ring of ALU pipelines operating on 64-bit scalar registers.
- **(b) [CORRECT]** Eight 2-dimensional physical tile registers (TMM0 to TMM7), each capable of holding up to 1,024 bytes (16 rows by 64 bytes), paired with a systolic grid of multiply-accumulate units executing TDPBF16PS or TDPBUSD instructions.
- **(c)** A dedicated optical co-processor connected over PCIe Gen 3.
- **(d)** An FPGA bitstream loaded into the L3 cache slice.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Intel AMX introduces eight 1KB tile registers (TMM0-TMM7) configured via the `ldtilecfg` instruction. The TMUL engine performs matrix multiplications directly on these 2D register tiles, processing 16 rows by 64 bytes per tile in hardware.

Question 03

In the Habana Gaudi 2 architecture, what is the exact aggregate bi-directional scale-out network bandwidth provided per accelerator chip using its integrated RoCE v2 ports?

- **(a)** 100 Gbps total
- **(b) [CORRECT]** 2.4 Tbps (24 ports * 100 GbE RoCE v2 integrated on-die)
- **(c)** 10 Gbps shared across the PCIe bus
- **(d)** 100 MB/s over USB 3.0

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Gaudi 2 integrates 24 standard 100GbE RoCE v2 engines natively on each accelerator die, providing 24 * 100 Gb/s = 2.4 Tbps (300 GB/s) of integrated networking bandwidth without requiring discrete external NICs.

Question 04

How does the Habana Gaudi 3 architecture improve compute and memory specifications over Gaudi 2?

- **(a)** It switches from silicon to vacuum tubes.
- **(b) [CORRECT]** It doubles MME matrix compute capacity (4 to 8 MMEs), expands TPCs from 8 to 64, increases HBM2e/HBM3 memory to 128 GB at 3.7 TB/s bandwidth, and upgrades the 24 on-die networking ports to 200 GbE RoCE.
- **(c)** It removes all on-die networking to save manufacturing costs.
- **(d)** It replaces HBM with standard DDR4 SODIMMs.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Gaudi 3 utilizes a dual-die package containing 64 Tensor Processor Cores (TPCs), 8 Matrix Multiplication Engines (MMEs), 128 GB of HBM with 3.7 TB/s bandwidth, and 24x 200GbE integrated RoCE ports, delivering 4x BF16 compute and 2x networking bandwidth over Gaudi 2.

Question 05

In OpenVINO Intermediate Representation (IR), what fundamental graph optimization is performed when an `Add` (bias) operation immediately follows a `MatMul` operation?

- **(a)** The model weights are rounded to zero.
- **(b) [CORRECT]** Operator Fusion: The OpenVINO Model Optimizer fuses the MatMul and Add nodes into a single composite `FullyConnected` primitive with a fused bias post-operation, eliminating an intermediate memory write and read cycle.
- **(c)** The two layers are executed on separate CPU cores concurrently.
- **(d)** The computational graph is converted into an audio waveform.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Operator fusion combines consecutive kernel operations into a single execution kernel. Fusing Bias-Add into MatMul allows the CPU/HPU to add the bias term directly into the hardware accumulation registers before writing the final result back to memory.

Question 06

When quantizing a pre-trained transformer model using Intel NNCF with Weight-Only INT4 Quantization, what is the role of the calibration dataset?

- **(a)** It trains the model for 100 additional epochs using AdamW.

- **(b) [CORRECT]** It passes representative domain token sequences through the model to record activation distributions across layers, allowing advanced algorithms (like AWQ or SmoothQuant) to detect activation outliers and determine optimal weight quantization scales and zero-points.
- **(c)** It checks student grades against the university database.
- **(d)** It deletes all layers with negative weights.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: Calibration sequences pass through the model without backpropagation. NNCF observes real activation dynamics, identifying high-magnitude outlier feature channels so that corresponding weight matrices can be scaled appropriately to preserve perplexity.

Question 07

Consider a linear layer with input dimension $d_{in} = 4,096$ and output dimension $d_{out} = 4,096$. In full fine-tuning, 16,777,216 parameters are updated. If LoRA is applied with rank $r = 16$, how many trainable parameters exist in this layer's adapter matrices A and B?

- **(a) [CORRECT]** 131,072 parameters (a 99.22% parameter reduction)
- **(b)** 1,048,576 parameters
- **(c)** 4,096 parameters
- **(d)** 8,388,608 parameters

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: LoRA decomposes the update into $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$. Parameters in $A = 16 \times 4,096 = 65,536$. Parameters in $B = 4,096 \times 16 = 65,536$. Total trainable parameters $= 65,536 + 65,536 = 131,072$. Reduction $= 1 - (131,072 / 16,777,216) = 99.22\%$.

Question 08

In a production Slurm cluster with 4 compute nodes, each containing 8 Habana Gaudi 2 accelerators (32 HPUs total), what is the optimal Slurm batch script directive configuration to launch a distributed PyTorch training job with 1 task per HPU?

- **(a)** `#SBATCH --nodes=1 --ntasks=1 --gres=gpu:1`
- **(b) [CORRECT]** `#SBATCH --nodes=4 --ntasks-per-node=8 --gres=habana:8`
- **(c)** `#SBATCH --nodes=32 --ntasks=1 --partition=debug`
- **(d)** `#SBATCH --cpus-per-task=1024 --mem=1TB`

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: To schedule across 4 distinct physical nodes with 8 accelerators each, `--nodes=4` specifies the node count, `--ntasks-per-node=8` launches 8 worker processes per node (1 per HPU), and `--gres=habana:8` requests 8 Habana HPUs per node.

Question 09

What is the primary operational purpose of the Intel Extension for PyTorch (IPEX) in deep learning workflows?

- **(a)** To replace PyTorch with Microsoft Excel macros.
- **(b) [CORRECT]** To provide drop-in hardware acceleration for PyTorch on Intel architectures, exposing automated mixed precision (BF16/INT8), graph fusions, weight prepacking, and native Intel AMX TMUL kernel execution.
- **(c)** To convert Python into JavaScript for web deployment.
- **(d)** To manage printer queues in the server room.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: IPEX extends PyTorch with optimizations designed specifically for Intel hardware, including vectorization via AVX-512, matrix acceleration via Intel AMX, memory layout format transformations (e.g., channels-last / blocked formats), and operator fusion.

Question 10

Why is the prompt processing phase ('prefill') in LLMs capable of achieving near-peak TFLOPs on Intel AMX or Gaudi MMEs, whereas the single-token autoregressive decoding phase achieves < 5% of peak TFLOPs?

- **(a)** Because decoding uses a secret unoptimized algorithm.
- **(b) [CORRECT]** Because prefill stacks all prompt tokens into a single large GEMM where arithmetic intensity is high (tokens re-use weights in cache/registers), whereas single-token decode is a matrix-vector GEMV where every single weight parameter must be loaded from DRAM to compute just 2 FLOPs per parameter.
- **(c)** Because AMX tiles automatically power off during decode.
- **(d)** Because prefill runs in C++, while decode runs in Python.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Arithmetic intensity of GEMM ($M \gg 1$) scales with batch/sequence length: $\frac{2MNK}{2(MK+KN+MN)} \approx K$, which easily exceeds the machine's arithmetic ridge point. For GEMV ($M = 1$), intensity is $\frac{2NK}{2(K+NK+N)} \approx 1$ FLOP/byte, saturating memory bandwidth while compute units idle.

Question 11

In the OpenVINO GenAI C++/Python API (`openvino_genai.LLMPipeline`), how does the pipeline manage KV-cache memory internally during multi-turn conversations?

- **(a)** It writes all KV-tensors to a temporary `.txt` file on disk.
- **(b) [CORRECT]** It manages stateful KV-cache tensors directly inside the OpenVINO compiled model runtime memory buffers, dynamically appending new token states and evicting expired states without expensive Python-to-C++ tensor marshal copies.
- **(c)** It deletes the KV-cache after every emitted character.
- **(d)** It requires the client browser to store the KV-cache in local cookies.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: `openvino_genai.LLMPipeline` uses stateful OpenVINO execution primitives where KV-cache tensors persist natively inside runtime memory. This bypasses the memory copying and overhead of extracting and re-injecting past key-values through Python dicts.

Question 12

When deploying an LLM with INT4 Weight-Only quantization, why is Symmetric Quantization computationally faster during runtime dequantization compared to Asymmetric Quantization?

- **(a)** Symmetric quantization requires no memory storage at all.
- **(b) [CORRECT]** In symmetric quantization, the zero-point $Z = 0$, meaning dequantization requires only a single multiplication: $x \approx q \times S$. Asymmetric quantization requires an extra subtraction: $x \approx (q - Z) \times S$, which adds instruction overhead and register pressure inside the tight inner loop.
- **(c)** Symmetric quantization can only be run on quantum processors.
- **(d)** Asymmetric quantization causes CPU cache lines to invalidate.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Evaluate`
> **Architectural Rationale**: Symmetric quantization constrains the real-world zero to map exactly to integer zero ($Z = 0$), simplifying the dequantization equation to $x = q \cdot S$. Asymmetric quantization carries a non-zero Z, requiring additional arithmetic operations per weight during on-the-fly dequantization.

Question 13

What is the primary role of the `TDPBF16PS` instruction in the x86-64 instruction set on Intel Sapphire Rapids processors?

- **(a)** To compute floating-point square roots.
- **(b) [CORRECT]** To perform a systolic dot-product of two pairs of Bfloat16 numbers from two 2D tile registers and accumulate the single-precision FP32 result into a destination 2D tile register.
- **(c)** To transfer data over the PCI Express bus.
- **(d)** To flush the CPU instruction pipeline.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `TDPBF16PS` (Tile Dot Product Bfloat16 Pairs to Single Precision Accumulator) is the core Intel AMX instruction that multiplies pairs of BF16 elements from two tile registers and accumulates them into 32-bit single-precision FP32 accumulators.

Question 14

In distributed LLM fine-tuning using DeepSpeed ZeRO-2 on a multi-node cluster, which components of the training state are partitioned across data-parallel ranks?

- **(a)** Only the student code comments.

- **(b) [CORRECT]** Both the Optimizer States (Adam momentum and variance) AND the Gradients are partitioned across all ranks, while model parameters are replicated.
- **(c)** The entire model parameters, gradients, and optimizer states are partitioned.
- **(d)** No states are partitioned; everything is duplicated.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: ZeRO-1 shards optimizer states (P_{os}). ZeRO-2 shards both optimizer states and gradients (P_{os+g}), reducing memory consumption by up to 8x compared to standard DDP while requiring no extra communication volume compared to standard all-reduce.

Question 15

What is the key advantage of Habana's integrated on-die RoCE networking over external InfiniBand PCIe adapters during distributed All-Reduce gradient synchronization?

- **(a)** It uses cheaper copper HDMI cables.
- **(b) [CORRECT]** It completely bypasses the PCIe bus, host root complex, and operating system kernel, streaming tensors directly from HBM memory to remote accelerator HBM memory via on-die RDMA network engines with sub-microsecond latency.
- **(c)** It eliminates the need for electric power in the server rack.
- **(d)** It automatically compresses training datasets.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Traditional GPU clusters must transfer tensors across the PCIe bus to a discrete Host Channel Adapter (HCA). Gaudi's integrated RoCE engines connect directly to the on-die crossbar, eliminating PCIe latency, host CPU overhead, and PCIe bandwidth bottlenecks.

Question 16

In continuous batching LLM serving architectures, why is Static Padding eliminated?

- **(a)** Because padding tokens are forbidden by the HTTP/2 specification.
- **(b) [CORRECT]** Because sequences are not combined into a single static 2D rectangular tensor; instead, requests are managed independently in dynamic token pools (e.g. via PagedAttention), evaluating only valid tokens at each decoding iteration.
- **(c)** Because transformers cannot compute attention on padding tokens.
- **(d)** Because tokenizers no longer use vocabulary IDs.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Static batching requires padding all sequences in a batch to the length of the longest sequence, wasting compute and memory on dummy PAD tokens. Continuous batching processes tokens dynamically from an active iteration queue, eliminating padding entirely.

Question 17

In the context of Intel NNCF, what is the effect of setting `group_size = -1` versus `group_size = 128` during INT4 Weight-Only quantization?

- **(a) [CORRECT]** `group_size = -1` enables per-channel quantization (one scale and zero-point for the entire channel), whereas `group_size = 128` partitions the channel into blocks of 128 elements, each with its own scale factor, significantly improving accuracy at a slight metadata memory cost.
- **(b)** `group_size = -1` deletes all weights with negative values.
- **(c)** `group_size = 128` crashes the OpenVINO runtime.
- **(d)** `group_size = -1` uses 1 bit per weight.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Evaluate`
> **Architectural Rationale**: In per-channel quantization (`group_size = -1`), outlier weights distort the quantization grid for the entire channel. Grouped quantization (`group_size = 128`) restricts the scale calculation to local 128-element windows, isolating outliers and preserving fine-grained weight distributions.

Question 18

In a FastAPI microservice serving an OpenVINO LLM, what is the purpose of running the model inference loop inside a dedicated background worker thread or process using `asyncio.to_thread()`?

- **(a)** To bypass user authentication checks.
- **(b) [CORRECT]** Because OpenVINO model inference is CPU-bound and synchronous; running it directly on the main asynchronous event loop would block the entire event loop, freezing all concurrent HTTP connections and health check probes.
- **(c)** To convert the server into a database.
- **(d)** To force the client to download the model `.bin` file.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: Python's asyncio runs a single-threaded cooperative event loop. If a synchronous, compute-heavy CPU function (like `compiled_model.generate()`) runs directly on the loop, it starves the loop, preventing it from accepting incoming socket connections or servicing concurrent requests.

Question 19

What is the role of the `alpha` hyperparameter in LoRA fine-tuning ($\Delta W = \frac{\alpha}{r} B \cdot A$)?

- **(a)** It sets the transparency level of the user interface.
- **(b) [CORRECT]** It is a constant scaling factor that stabilizes training dynamics when experimenting with different values of rank r, ensuring that the magnitude of the adapter update ΔW does not drastically fluctuate when the rank is changed.
- **(c)** It sets the number of CPU threads allocated to data loading.
- **(d)** It determines the learning rate decay schedule.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: The scaling factor $\frac{\alpha}{r}$ normalizes the magnitude of the adapter updates. Setting α to a constant (e.g. $2r$ or 16) allows practitioners to adjust rank r without having to completely retune the base learning rate.

Question 20

Why does a 13-billion parameter FP16 model require at least 26 GB of memory just to load its weights, and how does NNCF INT4 quantization alter this constraint?

- **(a) [CORRECT]** FP16 requires 2 bytes/param ($13 \times 10^9 \times 2 = 26$ GB); INT4 requires 0.5 bytes/param ($13 \times 10^9 \times 0.5 = 6.5$ GB), allowing a 13B model to fit comfortably into the memory of a single client workstation or edge server.
- **(b)** FP16 requires 16 bytes per parameter, whereas INT4 requires 4 bytes per parameter.
- **(c)** INT4 requires 52 GB of memory.
- **(d)** Quantization has no effect on memory footprint.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: 13 billion parameters $\times 2$ bytes $= 26$ GB. With INT4 (4 bits $= 0.5$ bytes), the weight footprint drops to $13 \times 0.5 = 6.5$ GB. Adding scaling metadata (~ 0.5 GB) totals ~ 7.0 GB, fitting into standard 16GB RAM devices.

Question 21

In Habana SynapseAI, what does the environment variable `PT_HPU_LAZY_MODE=0` do when debugging PyTorch code?

- **(a)** It speeds up training by 100x.
- **(b) [CORRECT]** It disables the lazy graph compiler and forces PyTorch to execute operations eagerly on the HPU, allowing developers to set standard Python breakpoints and inspect intermediate tensor values at the cost of lower throughput.
- **(c)** It powers off all server cooling fans.
- **(d)** It converts the code to run on an Intel Core i3 laptop.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In Lazy Mode (`PT_HPU_LAZY_MODE=1`), execution is deferred until tensors are materialized (e.g., printing or loss evaluation), making interactive debugging difficult. Eager mode (`PT_HPU_LAZY_MODE=0`) executes operations immediately, simplifying root-cause debugging of nan/inf errors.

Question 22

What is the primary difference between Hugging Face `PeftModel` and merging LoRA weights back into the base model using `model.merge_and_unload()`?

- **(a)** `PeftModel` can only run on Android; merged models can only run on iOS.

- **(b) [CORRECT]** `PeftModel` maintains base weights W_0 and adapter matrices A, B separately at runtime, computing two parallel forward paths; `merge_and_unload()` explicitly adds $\frac{\alpha}{r}(B \cdot A)$ directly into W_0 in memory, producing a single standalone model with zero inference latency overhead.
- **(c)** Merging weights deletes the attention mechanism.
- **(d)** Merged models require twice as much memory during inference.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: During fine-tuning, keeping W_0 and ΔW separate is essential. For production deployment, computing $W_{new} = W_0 + \frac{\alpha}{r}(B \cdot A)$ once eliminates the extra matrix multiply and activation addition at inference time, yielding zero latency penalty.

Question 23

How does Intel AMX accelerate INT8 quantization workloads using the `TDPBUSD` instruction?

- **(a)** By downloading pre-compiled weights from the internet.
- **(b) [CORRECT]** It performs dot products of signed bytes from one tile register with unsigned bytes from a second tile register, accumulating the intermediate products into 32-bit signed integer accumulators without numerical overflow.
- **(c)** By dividing two floating-point numbers.
- **(d)** By converting text tokens into Unicode binary.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `TDPBUSD` (Tile Dot Product of Signed and Unsigned Bytes with Accumulation) multiplies signed and unsigned 8-bit integers from two tile registers and accumulates them into 32-bit integer registers, enabling high-throughput INT8 matrix math.

Question 24

In Slurm job submission, what is the role of the environment variable `SLURM_PROCID`?

- **(a)** It specifies the university course code.
- **(b) [CORRECT]** It provides the global rank (process ID from 0 to N-1) of the current task within the active Slurm job step, used to initialize PyTorch distributed process groups (`rank=int(os.environ['SLURM_PROCID'])`).
- **(c)** It records the room temperature of the server room.
- **(d)** It sets the password for SSH login.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Slurm automatically populates `SLURM_PROCID` in the environment of every launched task. Distributed frameworks query this variable to determine their unique global communication rank.

Question 25

What is the technical cause of 'Perplexity Degradation' when quantizing an LLM to INT4 using naive Round-To-Nearest (RTN) without grouping or outlier protection?

- **(a)** The CPU runs out of disk storage space.
- **(b) [CORRECT]** Outlier weights and activations with large magnitudes force the uniform quantization scale factor to be excessively large, collapsing the vast majority of smaller weights into 0 or 1 quantization bin and destroying the subtle semantic representations of the network.
- **(c)** The Linux operating system kernel drops network packets.
- **(d)** The model produces outputs only in uppercase letters.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Evaluate`
> **Architectural Rationale**: In naive RTN, the scale factor is $S = \frac{\max(|W|)}{7}$. If a single weight outlier has magnitude 100 while 99.9% of weights lie in $[-1, 1]$, $S \approx 14.3$. Any weight between -7 and $+7$ rounds to 0, obliterating fine-grained representations.

Question 26

In OpenVINO runtime, what is the primary benefit of enabling dynamic shapes (`ov::Dimension()`) for LLM inference?

- **(a)** It forces the model to use 100% of the GPU at all times.
- **(b) [CORRECT]** It allows the compiled model to accept input prompts and context sequences of arbitrary variable lengths without requiring recompilation or padding to a fixed maximum sequence length.
- **(c)** It turns the neural network into an image classifier.
- **(d)** It disables the transformer feedforward layers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Fixed/static shapes require models to be compiled for specific sequence lengths (e.g., 512, 1024), forcing shorter prompts to be padded with dummy tokens. Dynamic shapes allow arbitrary batch and sequence dimensions to execute seamlessly.

Question 27

In Habana Gaudi's SynapseAI stack, what is the 'TPC LLVM Compiler'?

- **(a)** A web browser designed for supercomputers.
- **(b) [CORRECT]** A specialized backend compiler based on LLVM that compiles high-level C/C++ tensor kernels into VLIW SIMD machine code executed natively by the Tensor Processor Cores.
- **(c)** A database query optimizer.
- **(d)** A utility for formatting Linux ext4 partitions.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Gaudi's TPCs are fully programmable VLIW processors. The TPC LLVM compiler translates custom kernels (written in TPC-C) into microcode that controls the TPC's vector execution units, lookup tables, and addressing generators.

Question 28

What is the primary difference between GPTQ (Generalized Post-Training Quantization) and AWQ (Activation-aware Weight Quantization)?

- **(a) [CORRECT]** GPTQ uses second-order Taylor expansion (Hessian matrix inverse) to greedily quantize weights while updating remaining unquantized weights to compensate for quantization error; AWQ does not alter weight values, but identifies salient weight channels based on activation magnitudes and protects them with per-channel scaling.
- **(b)** GPTQ only works on image models; AWQ only works on audio.
- **(c)** AWQ requires retraining with SGD for 50 epochs; GPTQ does not.
- **(d)** GPTQ produces 32-bit floats; AWQ produces 64-bit doubles.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: GPTQ computes the inverse Hessian H^{-1} to find the optimal compensation update for remaining weights as each column is quantized. AWQ observes that protecting the top 1% of salient weights (multiplied by large activations) avoids error propagation without altering original weight coordinates.

Question 29

In an LLM benchmark report, what does the metric 'P99 Inter-Token Latency' represent?

- **(a)** The average token generation latency across all users.
- **(b) [CORRECT]** The 99th percentile threshold of latency between consecutive generated tokens; 99% of all emitted tokens experience an ITL equal to or faster than this value, revealing worst-case jitter and user-perceived stutter.
- **(c)** The percentage of tokens that contained spelling mistakes.
- **(d)** The total power consumption of the server rack.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: P99 latency measures tail performance. While average ITL might appear fast, periodic garbage collection, KV-cache re-allocation, or OS scheduling interruptions can cause noticeable stutters that show up in P99 spikes.

Question 30

What is 'FlashAttention' and what core hardware architectural mechanism does it exploit?

- **(a)** An Adobe Flash plugin for web browsers.
- **(b) [CORRECT]** An exact attention algorithm that tiles the computation of the softmax reduction across fast on-chip SRAM cache (L1/Shared Memory) to avoid materializing the full $N \times N$ intermediate attention matrix in slow HBM/DRAM, reducing memory I/O from quadratic to linear.
- **(c)** A method to flash new firmware onto hard drives.
- **(d)** A quantization scheme that rounds all weights to 0 or 1.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: FlashAttention computes scaled dot-product attention in blocks using online softmax normalization, keeping intermediate query, key, and value blocks in high-speed on-chip SRAM and eliminating redundant round-trips to high-latency DRAM/HBM.

Question 31

In a dual-socket Intel Xeon system with 64 cores per socket (128 total cores), how does configuring OpenVINO with two CPU streams (`ov::num_streams = 2`) mapped to NUMA nodes improve aggregate serving throughput?

- **(a)** It disables one of the CPU sockets to save power.
- **(b) [CORRECT]** It pins one inference execution stream exclusively to Socket 0 and its local DDR5 memory, and the second stream to Socket 1 and its local DDR5 memory, completely eliminating cross-socket UPI interconnect congestion.
- **(c)** It forces both sockets to execute the exact same token in lockstep.
- **(d)** It converts the CPU into an Ethernet router.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Binding execution streams to distinct NUMA nodes ensures that each stream accesses its local memory controller, doubling effective memory bandwidth and eliminating high-latency cross-socket bus transfers.

Question 32

In distributed LLM training, what is the role of 'Pipeline Parallelism' (PP)?

- **(a)** Running water pipes through the server rack for liquid cooling.
- **(b) [CORRECT]** Partitioning the sequential layers of a large neural network across multiple accelerators (e.g. layers 1-8 on Device 0, layers 9-16 on Device 1), passing intermediate activation tensors downstream during the forward pass and gradients upstream during the backward pass.
- **(c)** Executing Python scripts line by line.
- **(d)** Compressing training data before storing it on disk.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: When a model is too large to fit on a single device, Pipeline Parallelism divides layers across sequential devices. Techniques like 1F1B (One Forward, One Backward) minimize idle bubbles in the execution pipeline.

Question 33

In the Slurm workload manager, what does the command `sbatch run_gaudi.slurm` return upon successful execution?

- **(a)** The entire training dataset in CSV format.

- **(b) [CORRECT]** A confirmation message containing the unique assigned Slurm Job ID (e.g., `Submitted batch job 1048576`).
- **(c)** The root password for the compute nodes.
- **(d)** A compiled C++ binary.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `sbatch` submits the script to the Slurm controller queue asynchronously and prints the assigned Job ID, which can then be tracked via `squeue` or `sacct`.

Question 34

What is the primary motivation for using Asymmetric INT4 quantization on the output projections of LLM MLP feed-forward layers (e.g., `down_proj` or `fc2`)?

- **(a)** MLP layers do not contain weights.
- **(b) [CORRECT]** The inputs to these layers are post-activation features (e.g., SwiGLU or GELU outputs) that exhibit strictly non-negative or highly skewed value distributions where asymmetric zero-point offsets prevent precision loss.
- **(c)** Asymmetric quantization uses less disk space than symmetric quantization.
- **(d)** It allows the server to run without RAM.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Activations emerging from non-linearities like ReLU, GELU, or SwiGLU have asymmetric distributions (often non-negative). Using an asymmetric zero-point ensures the entire 16-level INT4 grid is dedicated to the active range rather than wasting half on empty negative space.

Question 35

What is the purpose of the `hl-smi -q` command in a Habana Gaudi server environment?

- **(a)** To format all hard drives.
- **(b) [CORRECT]** To query and display comprehensive XML/text telemetry including HPU board serial numbers, driver versions, PCIe bus IDs, memory usage, power consumption, clock frequencies, and thermal sensor readings.
- **(c)** To compile a PyTorch script into machine code.
- **(d)** To shut down the cluster.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Similar to `nvidia-smi -q`, the Habana Labs SMI query command reports detailed hardware, firmware, and telemetry data for all installed Gaudi HPUs.

Question 36

In a distributed deep learning cluster, what is the primary symptom of 'Straggler Nodes' during synchronous All-Reduce training?

- **(a)** The server room temperature drops to zero degrees.
- **(b) [CORRECT]** All healthy high-speed nodes complete their forward/backward compute steps quickly and sit idle waiting at the synchronization barrier for the single slowest node, causing overall cluster throughput to degrade to the speed of the straggler.
- **(c)** The model produces perfect validation accuracy on epoch 1.
- **(d)** The Slurm queue automatically cancels all user jobs.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: In synchronous DDP, no worker can begin the next optimization step until all workers have contributed their gradients to the All-Reduce collective. A single degraded node (due to thermal throttling, memory leaks, or packet loss) stalls the entire cluster.

Question 37

What is the primary role of the `AutoTokenizer` class from Hugging Face when deploying an LLM with OpenVINO?

- **(a)** To generate artificial images from text descriptions.
- **(b) [CORRECT]** To load the pre-trained token vocabulary, special token mappings (BOS, EOS, PAD), and subword merge rules, transforming raw input prompt strings into input ID tensors and attention masks expected by the OpenVINO model.
- **(c)** To convert Python into C++ code.
- **(d)** To monitor network bandwidth usage.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `AutoTokenizer` automatically matches the model's vocabulary and configuration, encoding text into numeric token IDs and decoding output token streams back into human-readable text.

Question 38

In the context of the Roofline Model, what does the 'Ridge Point' represent?

- **(a)** The physical height of the server rack in centimeters.
- **(b) [CORRECT]** The minimum arithmetic intensity (FLOPs/byte) at which an application can achieve the theoretical peak floating-point compute performance of the processor, calculated as $\frac{\text{Peak Compute Performance (GFLOPs/s)}}{\text{Peak Memory Bandwidth (GB/s)}}$.
- **(c)** The maximum temperature before the CPU shuts down.
- **(d)** The total number of memory channels on the motherboard.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: The ridge point separates the memory-bound region from the compute-bound region. If an algorithm's arithmetic intensity is below the ridge point, its performance is strictly capped by memory bandwidth.

Question 39

How does Intel NNCF implement 'SmoothQuant' to enable accurate 8-bit weight and activation quantization in LLMs?

- **(a)** By rounding all weights to even numbers.
- **(b) [CORRECT]** By applying a mathematically equivalent per-channel scaling transformation that migrates quantization difficulty from activations to weights ($W' = W \cdot \text{diag}(s)$, $X' = X \cdot \text{diag}(s)^{-1}$), smoothing activation outliers so both weights and activations can be quantized to INT8.
- **(c)** By increasing the learning rate by 10x.
- **(d)** By converting all matrices into diagonal matrices.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: SmoothQuant exploits the mathematical property that $Y = XW = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W)$. By choosing scaling vector s based on activation outlier magnitudes, activation peaks are smoothed out while weights absorb the scale, allowing 8-bit quantization.

Question 40

In Habana Gaudi systems, what does the term 'HBM2e' stand for, and why is it located on the same interposer package as the compute die?

- **(a)** Hard Block Memory 2 Enterprise; to save disk space.
- **(b) [CORRECT]** High Bandwidth Memory 2 Extended; mounted via a silicon interposer with a wide 1024-bit memory bus to provide multi-terabyte-per-second memory bandwidth with ultra-low trace capacitance and power consumption.
- **(c)** Hybrid Binary Matrix; to emulate quantum bits.
- **(d)** High Battery Module; to provide backup power during outages.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: HBM2e stacks DRAM dies vertically and connects them to the processor silicon via a silicon interposer with thousands of microscopic interconnects, enabling massive bandwidth (up to 2.45 TB/s on Gaudi 2) compared to narrow PCB memory buses.

Question 41

What is the primary function of the OpenVINO Model Server (OVMS) in an enterprise AI deployment?

- **(a) [CORRECT]** To provide a high-throughput, C++ based microservice supporting gRPC and REST APIs (compatible with KServe and Triton protocols) for hosting optimized OpenVINO and vLLM models in production Kubernetes environments.
- **(b)** To compile Python scripts into executable `.exe` files for Windows.
- **(c)** To back up server databases to tape drives.
- **(d)** To design user interface websites.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: OVMS is a scalable microservice written in C++ that exposes standardized gRPC and REST endpoints, offering dynamic batching, model versioning, and zero-overhead C++ execution for enterprise deployments.

Question 42

In a distributed training job using PyTorch FSDP (Fully Sharded Data Parallel), what is the purpose of 'Activation Checkpointing' (also known as Gradient Checkpointing)?

- **(a)** Saving student test scores to disk every 5 minutes.
- **(b) [CORRECT]** Discarding intermediate layer activations during the forward pass and recomputing them on-the-fly during the backward pass, trading ~20-30% extra compute time for a dramatic (up to 70%) reduction in activation memory.
- **(c)** Backing up the Linux kernel configuration to GitHub.
- **(d)** Writing logs to the console.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Normally, all intermediate activations must be stored in memory during the forward pass to compute gradients during the backward pass. Activation checkpointing discards most of them and recomputes them during the backward pass, preventing OOM errors on large batch sizes.

Question 43

What is the primary microarchitectural advantage of Bfloat16 over FP32 when executing GEMM operations on Intel AMX TMUL units?

- **(a) [CORRECT]** BF16 units deliver 2x higher arithmetic throughput per clock cycle compared to FP32, while consuming half the register and memory bandwidth, without sacrificing the FP32 dynamic exponent range.
- **(b)** BF16 uses zero electrical power.
- **(c)** BF16 can represent 100 times larger numbers than FP32.
- **(d)** BF16 does not require memory controllers.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Because BF16 elements are 16 bits (half the size of FP32), twice as many multiplier units can fit into the same silicon area. AMX TMUL delivers 1,024 BF16 operations per cycle per core, doubling compute throughput and halving memory traffic.

Question 44

In OpenVINO INT4 model serving, why is the memory footprint of the KV-cache often larger than the model weights themselves when serving large concurrency (e.g. batch size = 64, context length = 4,096 tokens)?

- **(a)** Because the KV-cache is intentionally padded with random numbers.
- **(b) [CORRECT]** Because the weights are statically compressed to INT4 (e.g. 3.5 GB for a 7B model), whereas the KV-cache grows linearly with batch size and context length in FP16 ($64 \times 4096 \times 32\ \text{layers} \times 4096\ \text{dim} \times 2\ \text{bytes} \approx 68.7$ GB), dwarfing the compressed weight footprint.
- **(c)** Because OpenVINO duplicates the KV-cache across all hard drives.
- **(d)** Because KV-cache cannot be stored in RAM.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Evaluate`
> **Architectural Rationale**: Model weights are constant regardless of user load ($O(1)$ with respect to concurrency). The KV-cache grows linearly ($O(B \cdot N)$) with batch size B and context length N. At high concurrency, KV-cache memory dominates total system RAM.

Question 45

Which of the following Slurm environment variables indicates the total number of compute nodes allocated to the active job?

- **(a) [CORRECT]** `SLURM_JOB_NUM_NODES`
- **(b)** `SLURM_TOTAL_DISKS`
- **(c)** `SLURM_RAM_PER_SOCKET`
- **(d)** `SLURM_USER_TIER`

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `SLURM_JOB_NUM_NODES` contains the total count of physical nodes assigned to the job allocation, commonly queried in bash scripts to compute global world size.

Question 46

In LLM generation sampling, what is the role of the 'Temperature' hyperparameter?

- **(a)** It regulates the physical cooling fans inside the server chassis.
- **(b) [CORRECT]** It scales the raw logit values before applying Softmax ($p_i = \frac{e^{z_i/T}}{\sum e^{z_j/T}}$), where $T < 1.0$ sharpens probabilities toward greedy high-confidence tokens, and $T > 1.0$ flattens probabilities toward higher diversity and randomness.
- **(c)** It sets the operating voltage of the processor.
- **(d)** It determines the font color of the output text.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Dividing logits by temperature T modulates the entropy of the probability distribution. As $T \to 0$, the distribution approaches an argmax (greedy decoding). High T creates a more uniform distribution, increasing lexical diversity.

Question 47

What is the primary reason why 'Continuous Batching' achieves up to 4x higher system throughput than 'Static Batching' in multi-user LLM serving benchmarks?

- **(a)** It overclocks the server memory bus.
- **(b) [CORRECT]** It eliminates bubbles and idle time by allowing completed requests to depart the batch immediately and admitting new incoming requests at every decode iteration, keeping accelerator compute and memory pipelines fully saturated.

- **(c)** It drops every fourth word generated by the model.
- **(d)** It runs the model in reverse order.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Static batching causes severe GPU underutilization because all requests in a batch are held hostage by the single longest generation request. Continuous batching evicts and injects requests dynamically at each token step, maximizing utilization.

Question 48

When converting a PyTorch model to OpenVINO IR using `ov.convert_model()`, how does setting `example_input` assist the conversion process?

- **(a)** It provides a training dataset to retrain the model weights.
- **(b) [CORRECT]** It enables PyTorch's `torch.jit.trace` engine to trace real tensor execution paths, deduce tensor shapes, and resolve dynamic control flow operations into a static computational graph.
- **(c)** It prints the model's licensing agreement to the console.
- **(d)** It measures the power consumption of the graphics card.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: `example_input` feeds a sample tensor through the PyTorch model graph so that TorchScript / FX tracer can observe exact operation sequences, input/output shapes, and layer connectivity.

Question 49

What is the primary technical distinction between Habana Gaudi 2 and contemporary GPU architectures regarding host CPU dependency?

- **(a)** Gaudi 2 requires 16 host CPUs per accelerator card.
- **(b) [CORRECT]** Gaudi 2 offloads the entire execution graph, DMA scheduling, and inter-accelerator collective communications (HCCL over on-die RoCE) directly to on-chip control processors, minimizing host CPU overhead and preventing host-side scheduling bottlenecks during multi-node scaling.
- **(c)** Gaudi 2 cannot communicate with host CPUs at all.
- **(d)** GPUs do not require host CPUs, while Gaudi requires 100 host CPUs.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Gaudi accelerators feature dedicated on-chip ARC control processors and integrated network engines that execute compiled SynapseAI graph recipes and HCCL collectives autonomously, freeing the host CPU from launching micro-kernels and managing interrupts.

Question 50

In an enterprise HPC cluster deployment, what is the ultimate goal of combining OpenVINO INT4 quantization on 5th Gen Intel Xeon Scalable CPUs with Habana Gaudi accelerators for distributed training?

- **(a)** To replace all cluster software with proprietary mainframe operating systems.
- **(b) [CORRECT]** To establish an end-to-end cost-effective, high-throughput AI infrastructure lifecycle: training and fine-tuning large foundation models efficiently at scale on Habana Gaudi clusters over standard RoCE Ethernet, and serving high-volume quantized inference directly on ubiquitous Intel Xeon server nodes with AMX acceleration without requiring expensive dedicated GPUs for every inference instance.
- **(c)** To run only legacy Fortran weather simulations.
- **(d)** To eliminate the need for network switches and data storage.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Evaluate`
> **Architectural Rationale**: This architecture forms a balanced enterprise strategy: Gaudi provides high-throughput, cost-effective distributed training over standard Ethernet fabrics, while Intel Xeon CPUs (accelerated by AMX TMUL and OpenVINO INT4) serve inference workloads with low TCO using existing data center compute capacity.
